SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA0305	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Y. Harsha Vardhan Reddy	**Reg. No.:**	192425227
Section / Batch:	2024 batch	**Date:**	13/08/26

Code of Conduct

I Y. Harsha Vardhan Reddy (192425227) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability. • Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

		No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as "She saw the man with a telescope." The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.
Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.
Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

ANSWERS

1. CFG Trees vs Dependency Parsing Structures

A language processing system must represent sentence structure for grammar checking and syntactic analysis. Two common approaches are **Context-Free Grammar (CFG) Trees** and **Dependency Parsing Structures**.

Analysis of CFG Trees

CFG represents a sentence as a **hierarchical structure of phrases** such as:

- S – Sentence
- NP – Noun Phrase
- VP – Verb Phrase
- Det – Determiner
- N – Noun
- V – Verb

Consider the sentence:

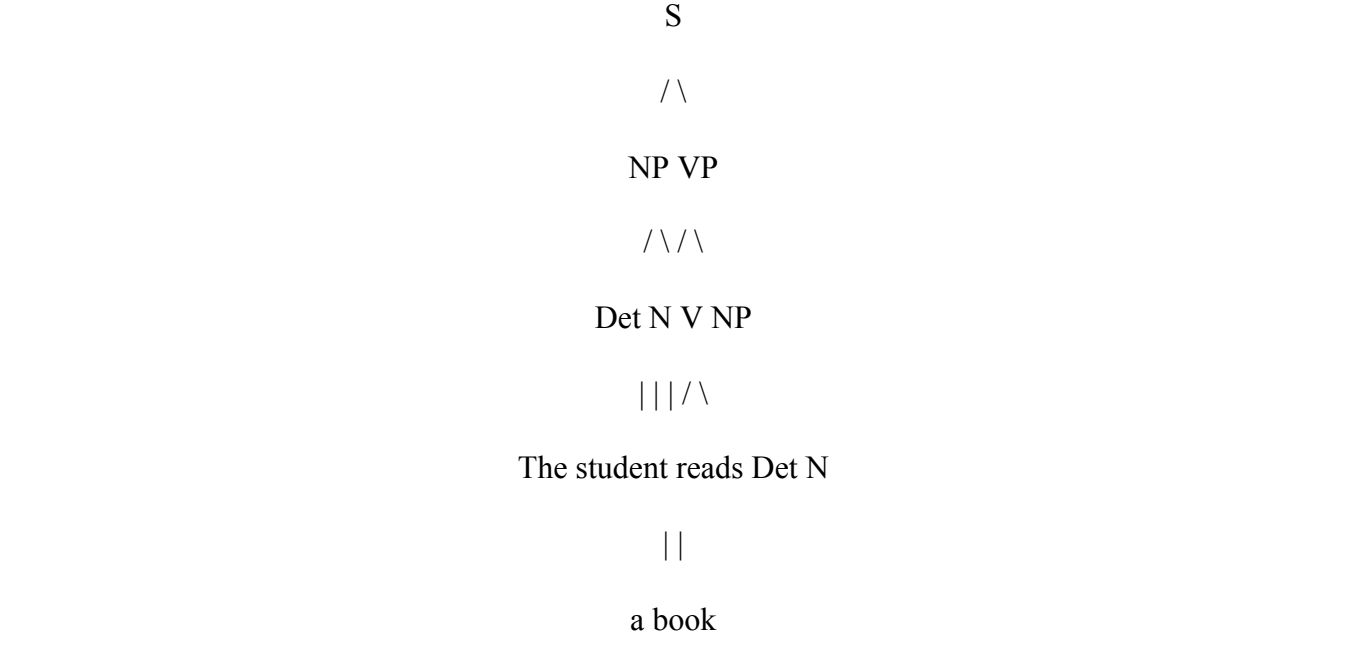
"The student reads a book."

A simple CFG can be:
S → NP VP

NP → Det N

VP → V NP

CFG Parse Tree

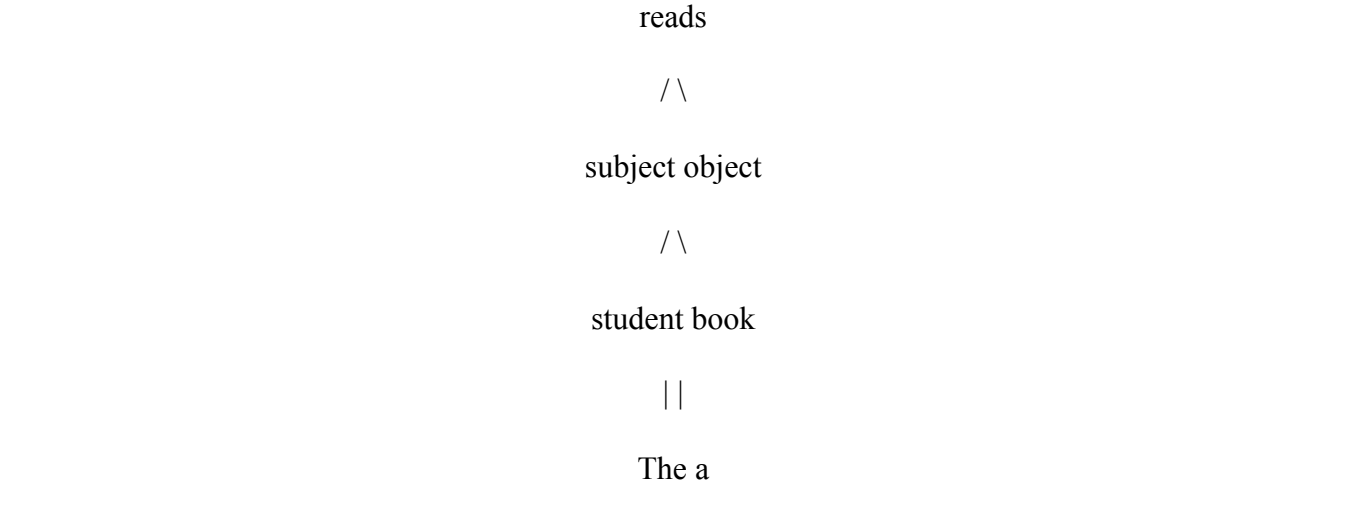


The CFG tree shows how words are grouped into phrases. For example, **"The student"** forms a noun phrase and **"reads a book"** forms a verb phrase.

Dependency Parsing Structure

Dependency parsing focuses on **direct relationships between words**.

For the same sentence:



Here:

- **reads** is the main head.
- **student** is related to reads as the subject.

- **book** is related to reads as the object.
- **The** depends on student.
- **a** depends on book.

Comparison Table

Feature	**CFG Trees**	**Dependency Parsing**
Representation	Hierarchical phrase structure	Direct relationships between words
Main focus	Phrases such as NP and VP	Head-dependent relationships
Structure	Tree of grammatical constituents	Tree of word dependencies
Word relationships	Indirect through phrases	Direct
Grammar checking	Good for phrase-level analysis	Good for relationship analysis
Long-distance relationships	More difficult to represent	Easier to represent
Main use	Syntactic and phrase analysis	Information extraction and NLP relationships

Justification

CFG trees are useful for understanding the **hierarchical phrase structure** of a sentence. However, Dependency Parsing is more effective for directly capturing relationships between words because it identifies the head and its dependents.

Conclusion: For capturing direct relationships between words, **Dependency Parsing is more effective**. CFG is preferred when phrase structure analysis is the main requirement.

2. Top-Down Parsing vs Earley Parsing

A real-time application must parse user input efficiently, even when the input is incomplete or ambiguous.

Top-Down Parsing

Top-down parsing begins from the **start symbol** and applies grammar rules to generate the input.

Example:

S

↓

NP VP

↓

Generate and match input words

The parser predicts a structure first and then attempts to match the sentence.
A major limitation is **backtracking**. If the parser selects an incorrect rule, it must return and try another rule. This can reduce performance when the input is ambiguous.

Earley Parsing

Earley Parsing is a dynamic parsing technique that uses three main operations.

Earley Parsing Operations

Operation	**Function**
Predictor	Predicts possible grammar rules
Scanner	Matches incoming words with grammar rules
Completer	Completes recognized grammatical structures

Earley Parsing can keep track of multiple possible parse structures while reading the input.

For example, for partial input:

“Book a flight to Delhi...”

the parser can begin processing immediately and continue when the user provides more

words. **Simple Earley Parsing Process**

Input

↓

Predictor

↓

Scanner

↓

Completer

↓

Possible Parse Structures

↓

Final Parse

Comparison Table

Feature	**Top-Down Parsing**	**Earley Parsing**
Starting point	Starts from the start symbol	Uses prediction, scanning, and completion
Parsing method	Predicts grammar structure first	Processes grammar and input dynamically
Backtracking	May be high	Reduced and avoids repeated work
Ambiguity handling	Difficult for many alternatives	Handles multiple possible parses
Incomplete input	Less suitable	More suitable
Incremental input	Limited	Supports incremental processing
Efficiency	Can be slow for ambiguous input	Efficient for general CFG parsing
Real-time use	Less suitable	More suitable

Justification:

Earley Parsing is more suitable for dynamic input conditions because it can:

- Handle ambiguous input.
- Process incomplete sentences.
- Continue parsing as more words arrive.
- Maintain multiple possible structures.
- Reduce unnecessary repeated parsing.

Conclusion: For a real-time application with incomplete and ambiguous input, **Earley Parsing is more suitable than Top-Down Parsing**.

3. CFG vs PCFG vs Neural Parsing for Ambiguity

Consider the sentence:

“She saw the man with a telescope.”

This sentence is structurally ambiguous.

The phrase **“with a telescope”** has two possible interpretations.

Interpretation 1: She used the telescope

She [saw with a telescope] [the man]

Here, **with a telescope** is connected to the action **saw**.

Meaning: She used a telescope to see the man.

Parse Structure 1

S

/ \

NP VP

| / \

She V VP

| / \

saw VP PP

| |

saw the man with a telescope

A simpler representation is:

saw

/ \

She with a telescope

\

man

Interpretation 2: The man had the telescope

She [saw [the man with a telescope]]

Here, **with a telescope** describes the noun **man**.

Meaning: She saw a man who had a telescope.

Parse Structure 2

S

/ \

NP VP

| / \

She V NP

| / \

saw NP PP

| |

the man with a telescope

Comparison of CFG, PCFG and Neural Parsing

Feature	**CFG**	**PCFG**	**Neural Parsing**
Basic approach	Rule-based grammar	Probabilistic grammar	Machine learning based
Ambiguity	Generates multiple valid parses	Selects the most probable parse	Useslearned context to select a parse
Probability	No	Yes	Learned automatically
Context handling	Limited	Better than CFG	Strong contextual understanding
Training data	Not required	Required for probabilities	Requires large training data
Accuracy	Moderate	Higher	Usually high in real world applications
Adaptability	Low	Moderate	High
Real-world use	Limited	Useful and interpretable	Highly effective

Analysis

CFG:
CFG can generate both valid structures but cannot decide which meaning is intended.

PCFG:
PCFG assigns probabilities to grammar rules and selects the most likely parse.

For example:

VP → VP PP [Probability]

NP → NP PP [Probability]

The parse with the higher probability is selected.

Neural Parsing:
Neural parsing learns language patterns and contextual relationships from training data. It can use surrounding information to select the most appropriate interpretation.

Justification

For real-world applications, **Neural Parsing is generally the most effective** because it handles complex patterns and contextual ambiguity better.

However:

- CFG is suitable for simple rule-based systems.
- PCFG provides a balance between probability-based accuracy and grammatical interpretability.
- Neural Parsing provides stronger performance for large-scale modern NLP applications.

Conclusion: For real-world ambiguity handling, **Neural Parsing is the most effective approach**.

4. Feature Structures vs Subcategorization Frames

A grammar system must handle **subject–verb agreement** and **verb argument structures**.

Feature Structures

Feature Structures store grammatical properties of words.

Common features include:

- Number
- Person
- Gender
- Tense

For example:

“The student writes.”

Feature Structure Representation

Feature	**Subject: Student**	**Verb: Writes**

Number	Singular	Singular
Person	Third Person	Third Person
Tense	—	Present

Since both subject and verb are singular, the sentence is grammatically correct.

Student [Number = Singular]

Writes [Number = Singular]

Agreement = Correct ✔

An incorrect example is:

“The student write.” ✘

Here, the singular subject does not match the expected singular verb form.

Subcategorization Frames
Subcategorization Frames define the arguments that a verb requires or allows.

For example:

Give → Subject + Direct Object + Indirect Object

Sentence:

“The teacher gives the student a book.”

The verb **gives** requires a subject and can take both an indirect and direct

object. **Subcategorization Frame Table**

Verb	**Required Arguments**	**Example**
Give	Subject + Direct Object + Indirect Object	Teacher gives student a book
Sleep	Subject	Student sleeps
Recommend	Subject + Action	Doctor recommends treatment
Put	Subject + Object + Location	She puts the book on the table

Comparison Table

Feature	Feature Structures	Subcategorization Frames
Main purpose	Represents grammatical features	Represents verb argument requirements
Agreement handling	Strong	Limited
Subject–verb agreement	Directly supported	Not the main purpose
Number and person	Directly represented	Usually not directly represented
Verb arguments	Limited support	Strong support
Object requirements	Limited	Clearly definesrequired arguments
Main use	Grammatical correctness	Verb and entity relationship analysis

Combined Structure

The best grammar system can combine both methods:

Feature Structures

+

Subcategorization Frames

↓

Subject-Verb Agreement

+

Correct Verb Arguments

↓

Better Grammatical Analysis

Justification

Feature Structures provide better support for **subject–verb agreement and grammatical correctness** because they directly represent features such as number and person.

Subcategorization Frames provide better support for **verb argument structures** because they specify the required objects or entities associated with a verb.

Conclusion: Feature Structures are better for enforcing agreement, while Subcategorization Frames are better for validating verb arguments. **Combining both provides the best overall grammatical analysis.**

5. Transition-Based vs Graph-Based Dependency Parsing

A large-scale NLP system must perform fast and accurate dependency parsing on big datasets.

The two approaches are **Transition-Based Parsing** and **Graph-Based Parsing**.

Transition-Based Parsing

Transition-Based Parsing constructs a dependency tree step by step using actions such as:

- Shift
- Reduce
- Left-Arc
- Right-Arc

Process Diagram

Input Sentence

↓

SHIFT

↓

LEFT-ARC / RIGHT-ARC

↓

REDUCE

↓

Dependency Tree

The parser makes a sequence of local decisions to build the final dependency tree.

Advantage: It is generally fast and suitable for processing a large number of sentences.

Limitation: An incorrect early decision can affect later decisions.

Graph-Based Parsing

Graph-Based Parsing represents possible relationships between words as a graph.

It scores possible dependency edges and selects the best overall dependency tree.

Process Diagram

Input Sentence

↓

Generate Possible Dependencies

↓

Assign Scores to Dependency Edges

↓

Global Optimization

↓

Select Best Dependency Tree

Advantage: It considers global relationships and can provide strong accuracy.

Limitation: It can require more computation than transition-based parsing.

Comparison Table

Feature	**Transition-Based Parsing**	**Graph-Based Parsing**
Decision-making	Sequence of local actions	Global optimization of dependency tree
Parsing process	Incremental	Evaluates possible relationships
Main approach	Shift, Reduce and Arc actions	Scores dependency edges
Speed	Usually very fast	More computationally expensive
Accuracy	Good	Often strong with global information
Error propagation	Earlier errors can affect later decisions	Less dependent on previous local decisions
Long-distance dependencies	Can be more difficult	Better global consideration
Large-scale processing	Highly suitable	Suitable when accuracy is prioritized
Real-time use	Very suitable	May require more computation

Decision-Making Comparison

Transition-Based:

Input → Local Decision → Local Decision → Dependency Tree

Graph-Based:

Input → All Possible Relationships → Global Scoring → Best Tree

Justification

For **large-scale applications where speed and high-volume processing are the main requirements**, Transition-Based Parsing is generally more suitable because:

- It is fast.
- It processes sentences incrementally.
- It requires efficient local decisions.
- It is suitable for real-time and large datasets.

Graph-Based Parsing is preferred when **global analysis and maximum parsing accuracy** are more important than computational cost.

Final Requirement Table

Requirement	**More Suitable Method**
Very fast parsing	Transition-Based Parsing
Real-time processing	Transition-Based Parsing
Large datasets	Transition-Based Parsing
Global relationship analysis	Graph-Based Parsing
Global optimization	Graph-Based Parsing
High accuracy priority	Graph-Based Parsing

Final Conclusion

For the given requirement of fast and accurate dependency parsing on large datasets, Transition-Based Parsing is generally more suitable for large-scale applications because of its high speed and efficient incremental decision-making. However, Graph-Based Parsing is a better choice when global relationships and higher overall accuracy are the main priority.

Rubrics for Calculation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Scope of Items	20%	Selects highly relevant items with clearscope and	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant

		justification			
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director

Signature:	Signature:	Signature:
Date:	Date:	Date: